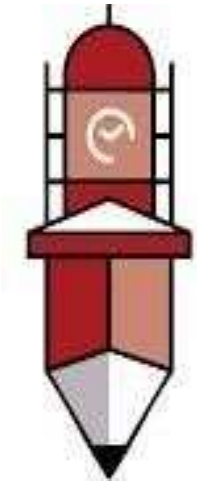


The University of Faisalabad

LABMANUAL

Name:

HAMMAD
SHAD

Reg No#: 2023-BS-AI-146

Department:
BS-Artificial Intelligence 6TH **A**

Subject:
Data Mining

Submitted To:
SIR ARHUM

Department of Computer Science
The University Of Faisalabad

MAY2026

TABLEOFCONTENT

SR NO:	DATE	TOPIC
Lab1	**29/01/26**	Introductionto Data Mining,Software Introduction.
Lab2	**19/02/26**	CRISP-DM process,Types of data,Real-world data mining applications.
Lab3	**26/02/26**	Handling missing values,emoving duplicates,Noise handling using Python.
Lab4	**05/03/26**	Normalization,Standardization,Discretization,Summary Statistics.
Lab5	**12/03/26**	Market basket analysis,Support,Confidence,Lift calculation manually.
Lab6	**26/03/26**	Apriori algorithm using Python,Frequent itemset generation
Lab7	**02/04/26**	Implement Naïve Bayes and KNN
Lab8	**09/04/26**	K-Means clustering
Lab9	**23/04/26**	Hierarichal clustering
Lab10	**30/04/26**	Distance based outliers,Isolation forest,Practical Dataset
Lab11	**07/05/26**	FP-Tree construction, FP-Growth implementation using library
Lab12	**14/05/26**	Introduction to sentiment analysis

Lab-1

Introduction to Data Mining

Data Mining is the process of extracting useful information and hidden patterns from large amounts of data. It helps in finding meaningful knowledge from databases using different techniques and algorithms. Data Mining is widely used in business, healthcare, banking, education, marketing, and many other fields.

The main purpose of Data Mining is to analyze data and make better decisions. It can identify trends, relationships, and patterns that are difficult to find manually.

Nowadays, companies collect a huge amount of data every day. Data Mining helps them understand customer behavior, improve services, and increase business performance.

APPLICATIONS OF DATA MINING

- Fraud Detection in banks
- Recommendation Systems in online shopping
- Disease Prediction in healthcare
- Student Performance Analysis in education
- Customer Behavior Analysis in business
- Social Media Data Analysis

TYPES OF DATASETS

1. **Structured Dataset :** Data stored in rows and columns like Excel sheets or databases.
2. **Unstructured Dataset** : Data without proper format such as images, videos, and text files.
3. **Semi-Structured Dataset** : Data partially organized like XML and JSON files.

GOOGLE COLAB

Google Colab is an online platform used for writing and running Python code. It is very useful for Data Mining and Machine Learning projects because it provides free GPU support and does not require software installation.

Advantages of Google Colab

- Easy to use
- Free cloud environment
- Supports Python libraries
- Can save work directly on Google Drive
- Useful for Machine Learning and Data Analysis

DATA MINING PROCESS

The Data Mining process consists of different steps:

1. **Data Collection** : Gathering data from different sources.
2. **Data Cleaning** : Removing errors, duplicate values, and missing data.
3. **Data Integration** : Combining data from multiple sources.
4. **Data Transformation** : Converting data into a suitable format for analysis.
5. **Data Mining**: Applying algorithms to find patterns and relationships.
6. **Evaluation:** Checking the accuracy and usefulness of results.
7. **Knowledge Representation:**Presenting results in graphs, charts, or reports.

FEATURES OF DATA MINING

- Helps in decision-making
- Saves time and effort
- Finds hidden patterns in data
- Improves business performance
- Supports prediction and analysis
- Handles large datasets efficiently

Common Python Libraries

- NumPy
- Pandas
- Matplotlib
- Scikit-learn

These libraries help in data analysis, visualization, and machine learning tasks.

ADVANTAGES OF DATA MINING

- Improves business strategies
- Helps in future prediction
- Detects fraud and risks
- Increases customer satisfaction
- Supports scientific research

DISADVANTAGES OF DATA MINING

- Privacy and security issues
- Requires large storage space
- Can produce inaccurate results if data quality is poor
- Complex algorithms may require high processing power

Lab-2

CRISP-DM PROCESS

CRISP-DM stands for Cross Industry Standard Process for Data Mining. It is a popular process model used in Data Mining projects. It provides a step-by-step approach to solve data-related problems and helps in organizing the complete workflow of a project.

PHASES OF CRISP-DM

1. Business Understanding

In this phase, the problem and project goals are identified. The organization decides what they want to achieve through Data Mining.

Example:
A company wants to predict customer buying behavior.

2. Data Understanding

Data is collected and analyzed to understand its quality and structure.

Tasks include:

- Collecting data
- Identifying missing values
- Understanding dataset features

3. Data Preparation

The collected data is cleaned and transformed into a suitable format for analysis.

Activities:

- Removing duplicate values
- Handling missing data
- Normalizing and organizing data

4. Modeling:

Different algorithms and machine learning models are applied to the prepared dataset.

Examples:

- Decision Trees
- Regression
- Clustering

4. Evaluation:

The performance of the model is checked to see whether it meets the project objectives.

Example:
Checking prediction accuracy and error rate.

6. Deployment

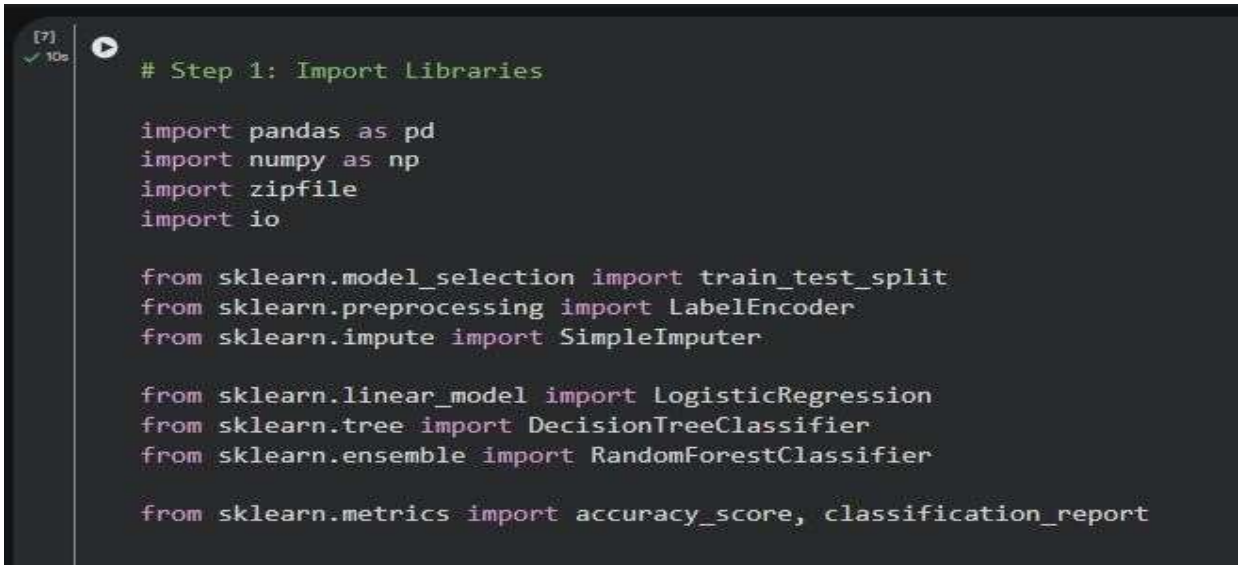
The final model is implemented in a real system for practical use.

Example:
Using a recommendation system in an online shopping website.

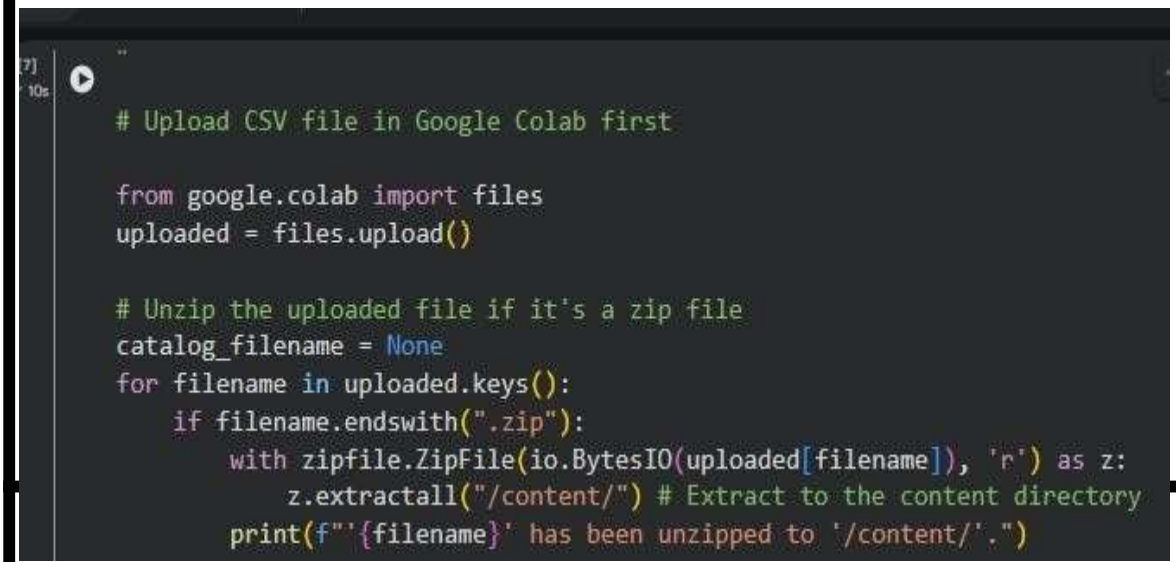
TASK:

Apply CRISP-DM steps on a sample dataset (e.g., student performance dataset).

1. **Define business objective: Predict student pass/fail.**
2. **Explore the dataset.**
3. **Clean missing values.**
4. **Build a classification model.**
5. **Evaluate accuracy.**
6. **Suggest a deployment plan**



```
[7]
10s
# Step 1: Import Libraries

import pandas as pd
import numpy as np
import zipfile
import io

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report
```



```
[7]
10s
# Upload CSV file in Google Colab first

from google.colab import files
uploaded = files.upload()

# Unzip the uploaded file if it's a zip file
catalog_filename = None
for filename in uploaded.keys():
    if filename.endswith(".zip"):
        with zipfile.ZipFile(io.BytesIO(uploaded[filename]), 'r') as z:
            z.extractall("/content/") # Extract to the content directory
        print(f"'{filename}' has been unzipped to '/content/'.")
```

```
# Step 3: Business Objective
# ============================================

print("\nBusiness Objective:")
print("Predict whether student will Pass or Fail")

# ============================================
# Step 4: Data Understanding
# ============================================

print("\nDataset Information")
print(df.info())

print("\nMissing Values")
print(df.isnull().sum())

print("\nDataset Shape")
print(df.shape)
```

```
# ============================================
# Step 5: Create Pass/Fail Target
# ============================================

# Assuming marks column name is Final_Grade

df['Result'] = np.where(df['G3'] >= 10, 1, 0)

# 1 = Pass
# 0 = Fail

print("\nResult Column Added")
print(df[['G3', 'Result']].head())

# ============================================
# Step 6: Handle Missing Values
# ============================================

# Separate numerical and categorical columns

num_cols = df.select_dtypes(include=np.number).columns
cat_cols = df.select_dtypes(exclude=np.number).columns

# Fill numerical missing values with mean

num_imputer = SimpleImputer(strategy='mean')
```

```
# ============================================
# Step 7: Encode Categorical Data
# ============================================

label_encoder = LabelEncoder()

for col in cat_cols:
    df[col] = label_encoder.fit_transform(df[col])

# ============================================
# Step 8: Define Features and Target
# ============================================

X = df.drop(['Result'], axis=1)
y = df['Result']

# ============================================
# Step 9: Split Dataset
# ============================================

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
# Step 10: Build Logistic Regression Model
# ==========================================

lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(X_train, y_train)

lr_pred = lr_model.predict(X_test)

lr_accuracy = accuracy_score(y_test, lr_pred)

print("\nLogistic Regression Accuracy")
print(lr_accuracy)

# ==========================================
# Step 11: Build Decision Tree Model
# ==========================================

dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_pred)
```

ables Terminal

```
# ==========================================
# Step 14: Monitor Model Performance
# ==========================================

print("\nModel Performance Summary")

print("Logistic Regression Accuracy:", lr_accuracy)
print("Decision Tree Accuracy:", dt_accuracy)
print("Random Forest Accuracy:", rf_accuracy)

# ==========================================
# Step 15: Deployment Plan
# ==========================================

print("\nDeployment Plan")

print("""
1. Save trained model
2. Deploy model on school management system
3. Input student information
4. Predict Pass or Fail automatically
5. Monitor model performance regularly
6. Update model with new student data
""")
```

```
[7]
✓ 10s
```

```
6. Update model with new student data
"""
```

```
Choose Files  student_data.csv
student_data.csv(text/csv) - 41983 bytes, last modified: 5/20/2026 - 100% done
Saving student_data.csv to student_data (2).csv
First 5 Rows
  school sex  age address famsize Pstatus  Medu  Fedu     Mjob      Fjob  ... \
0     GP   F   18       U     GT3       A     4     4  at_home   teacher  ...
1     GP   F   17       U     GT3       T     1     1  at_home     other  ...
2     GP   F   15       U     LE3       T     1     1  at_home     other  ...
3     GP   F   15       U     GT3       T     4     2   health  services  ...
4     GP   F   16       U     GT3       T     3     3    other     other  ...

   famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0       4        3      4     1     1      3        6   5   6   6
1       5        3      3     1     1      3        4   5   5   6
2       4        3      2     2     3      3       10   7   8  10
3       3        2      2     1     1      5        2  15  14  15
4       4        3      2     1     2      5        4   6  10  10

[5 rows x 33 columns]

Business Objective:
Predict whether student will Pass or Fail

Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
```

```
nursery       0
higher        0
internet      0
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
absences      0
G1            0
G2            0
G3            0
Result        0
dtype: int64

Training Data Shape: (316, 33)
Testing Data Shape: (79, 33)

Logistic Regression Accuracy
1.0

Decision Tree Accuracy
1.0

Random Forest Accuracy
1.0

Classification Report
              precision    recall  f1-score   support

         0.0       1.00      1.00      1.00        27
         1.0       1.00      1.00      1.00        52

    accuracy                           1.00        79
   macro avg       1.00      1.00      1.00        79
```

```
Decision Tree Accuracy
1.0

Random Forest Accuracy
1.0

Classification Report
              precision    recall  f1-score   support

         0.0       1.00      1.00      1.00        27
         1.0       1.00      1.00      1.00        52

    accuracy                           1.00        79
   macro avg       1.00      1.00      1.00        79
weighted avg       1.00      1.00      1.00        79


Model Performance Summary
Logistic Regression Accuracy: 1.0
Decision Tree Accuracy: 1.0
Random Forest Accuracy: 1.0

Deployment Plan

1. Save trained model
2. Deploy model on school management system
3. Input student information
4. Predict Pass or Fail automatically
5. Monitor model performance regularly
6. Update model with new student data
```

Lab-3

Handling missing values,emoving duplicates,Noise handling using Python.

Handling Missing Values:

Missing values are empty or null values present in a dataset. These missing values can affect the accuracy of machine learning models. Therefore, they should be handled properly before training the model.

Methods of Handling Missing Values

- Fill missing numerical values with Mean or Median
- Fill categorical values with Mode
- Remove rows containing missing values

Removing Duplicates

Duplicate records are repeated rows in a dataset. They increase data size and can produce incorrect analysis results.

Advantages of Removing Duplicates

- Improves data quality
- Reduces dataset size
- Gives accurate results

Noise Handling using Python

Noise in data means unwanted or incorrect values that reduce data quality. Noise can occur due to human errors, machine errors, or incorrect data collection.

Methods of Noise Handling

1. Smoothing
2. Removing Outliers
3. Data Filtering

PRACTICAL IMPLEMENTATION

```
import pandas as pd
import numpy as np

data = {
    'Name': ['Ali', 'Sara', 'Ahmed', 'Sara', 'Zain', 'Ali', None],
    'Age': [20, 21, np.nan, 21, 120, 20, 22],
    'Marks': [85, 90, 95, 90, 400, 85, np.nan]}
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
```

```
Original Dataset:
    Name    Age  Marks
0    Ali   20.0   85.0
1   Sara   21.0   90.0
2  Ahmed    NaN   95.0
3   Sara   21.0   90.0
4   Zain  120.0  400.0
5    Ali   20.0   85.0
```

```
[15] ✓ 0s
print("\nMissing Values:")
print(df.isnull())
print("\nTotal Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
    Name    Age  Marks
0  False  False  False
1  False  False  False
2  False   True  False
3  False  False  False
4  False  False  False
5  False  False  False
6   True  False   True

Total Missing Values:
Name     1
Age      1
Marks    1
dtype: int64
```

```
[24] ✓ 0s
df_removed = df.dropna()
print("\nDataset After Removing Missing Values:")
print(df_removed)
```

```
Dataset After Removing Missing Values:
    Name        Age  Marks Marks_Category
0    Ali  20.000000   85.0          High
1   Sara  21.000000   90.0          High
2  Ahmed  37.333333   95.0          High
3   Sara  21.000000   90.0          High
```

```
[17] ✓ 0s
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Marks'].fillna(df['Marks'].mean(), inplace=True)
```

```
/tmp/ipykernel_4226/622255692.py:1: FutureWarning: A value is trying to be set on a copy o
The behavior will change in pandas 3.0. This inplace method will never work because the in

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col:


  df['Age'].fillna(df['Age'].mean(), inplace=True)
/tmp/ipykernel_4226/622255692.py:2: FutureWarning: A value is trying to be set on a copy o
The behavior will change in pandas 3.0. This inplace method will never work because the in

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col:


  df['Marks'].fillna(df['Marks'].mean(), inplace=True)
```

```
[18] ✓ 0s
df['Name'].fillna(df['Name'].mode()[0], inplace=True)
print("\nDataset After Replacing Missing Values:")
print(df)
```

```
Dataset After Replacing Missing Values:
    Name        Age      Marks
0    Ali  20.000000  85.000000
1   Sara  21.000000  90.000000
2  Ahmed  37.333333  95.000000
3   Sara  21.000000  90.000000
4   Zain 120.000000 400.000000
5    Ali  20.000000  85.000000
6    Ali  22.000000 140.833333
/tmp/ipykernel_4226/1579856853.py:1: FutureWarning: A value is trying to be set on a copy
```

```
[19] ✓ 0s
print("\nDuplicate Rows:")
print(df.duplicated())
df_no_duplicates = df.drop_duplicates()
print("\nDataset After Removing Duplicates:")
print(df_no_duplicates)
```

```
Duplicate Rows:
0    False
1    False
2    False
3     True
4    False
5     True
6    False
dtype: bool

Dataset After Removing Duplicates:
    Name        Age      Marks
0    Ali  20.000000  85.000000
1   Sara  21.000000  90.000000
2  Ahmed  37.333333  95.000000
4   Zain 120.000000 400.000000
6    Ali  22.000000 140.833333
```

```
[23] ✓ 0s
Q1 = df['Marks'].quantile(0.25)
Q3 = df['Marks'].quantile(0.75)
IQR = Q3 - Q1
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
outliers = df[(df['Marks'] < lower_limit) |
              (df['Marks'] > upper_limit)]
print("\nOutliers:")
print(outliers)
```

```
Outliers:
   Name    Age  Marks Marks_Category
4  Zain  120.0  400.0           NaN
```

```
[21] ✓ 0s
df_no_outliers = df[(df['Marks'] >= lower_limit) &
                    (df['Marks'] <= upper_limit)]
print("\nDataset After Removing Outliers:")
print(df_no_outliers)
```

```
Dataset After Removing Outliers:
    Name        Age      Marks
0    Ali  20.000000  85.000000
1   Sara  21.000000  90.000000
2  Ahmed  37.333333  95.000000
3   Sara  21.000000  90.000000
5    Ali  20.000000  85.000000
6    Ali  22.000000 140.833333
```

```
6    Ali  22.000000  140.833333
```

```
[22]
✓ 0s
bins = [0, 60, 80, 100]
labels = ['Low', 'Medium', 'High']
df['Marks_Category'] = pd.cut(df['Marks'],
                              bins=bins,
                              labels=labels)
print("\nDataset After Binning:")
print(df)
```

```
Dataset After Binning:
    Name         Age       Marks Marks_Category
0    Ali   20.000000   85.000000          High
1   Sara   21.000000   90.000000          High
2  Ahmed   37.333333   95.000000          High
3   Sara   21.000000   90.000000          High
4   Zain  120.000000  400.000000           NaN
5    Ali   20.000000   85.000000          High
6    Ali   22.000000  140.833333           NaN
```

TASK:

```
[26]
✓ 0s
import pandas as pd
import numpy as np
data = {
    'StudentID': [101,102,103,104,105,106,107,108,109,110,111,112],
    'Name': ['Aisha','Ali','Sara','Ahmed','John','Aisha','David','Emma','Noah','Olivia','Liam','Sophia'],
    'Age': [20,19,21,22,18,20,-1,23,200,20,19,np.nan],
    'Gender': ['Female','Male','Female','Male','Male','Female','Male','Female','Male','Female','Male','Female'],
    'Marks': [85,78,95,105,67,85,45,150,56,72,'abc',90],
    'Attendance': [92,88,np.nan,85,110,92,75,98,80,np.nan,85,89],
    'Email': [
        'aisha@gmail.com',
        'ali@gmail.com',
        'sara@gmail.com',
        'ahmed@gmail.com',
        'john@gmail.com',
        'aisha@gmail.com',
        'david@gmail.com',
        'emma@gmail.com',
        'noah@gmail.com',
        'olivia@gmail.com',
        'liam@gmail.com',
        'sophia@gmail.com'
    ]
}
```

```
df = pd.DataFrame(data)

print("First 5 Rows:")
print(df.head())
print("\nData Types:")
print(df.dtypes)
```

```
First 5 Rows:
   StudentID   Name   Age  Gender Marks  Attendance          Email
0        101  Aisha  20.0  Female    85        92.0  aisha@gmail.com
1        102    Ali  19.0    Male    78        88.0    ali@gmail.com
2        103   Sara  21.0  Female    95         NaN   sara@gmail.com
3        104  Ahmed  22.0    Male   105        85.0  ahmed@gmail.com
4        105   John  18.0    Male    67       110.0   john@gmail.com

Data Types:
StudentID       int64
Name           object
Age           float64
Gender         object
Marks          object
Attendance    float64
Email          object
dtype: object
```

```
print("\nMissing Values Count:")
print(df.isnull().sum())
df['Marks'] = pd.to_numeric(df['Marks'], errors='coerce')
print("\nMarks Column After Conversion:")
print(df['Marks'])
marks_avg = df['Marks'].mean()
df['Marks'].fillna(marks_avg, inplace=True)
print("\nMarks After Filling Missing Values:")
print(df['Marks'])
```

```
Missing Values Count:
StudentID     0
Name          0
Age           1
Gender        0
Marks         0
Attendance    2
Email         0
dtype: int64

Marks Column After Conversion:
0      85.0
1      78.0
2      95.0
3     105.0
4      67.0
5      85.0
6      45.0
7     150.0
8      56.0
9      72.0
10      NaN
11     90.0
```

```python
attendance_median = df['Attendance'].median()
df['Attendance'].fillna(attendance_median, inplace=True)
print("\nAttendance After Filling Missing Values:")
print(df['Attendance'])
age_mean = df['Age'].mean()
df['Age'].fillna(age_mean, inplace=True)
print("\nAge After Filling Missing Values:")
print(df['Age'])
print("\nDuplicate Rows:")
print(df[df.duplicated()])
print("\nDataset Size Before Removing Duplicates:")
print(df.shape)
df = df.drop_duplicates()
```

```
Attendance After Filling Missing Values:
0     92.0
1     88.0
2     88.5
3     85.0
4    110.0
5     92.0
6     75.0
7     98.0
8     80.0
9     88.5
10    85.0
11    89.0
```

```python
# Remove invalid age values
df = df[(df['Age'] >= 0) & (df['Age'] <= 100)]
# Remove invalid marks values
df = df[(df['Marks'] >= 0) & (df['Marks'] <= 100)]
# Remove invalid attendance values
df = df[(df['Attendance'] >= 0) & (df['Attendance'] <= 100)]
```

```python
Q1_age = df['Age'].quantile(0.25)
Q3_age = df['Age'].quantile(0.75)
IQR_age = Q3_age - Q1_age
lower_age = Q1_age - 1.5 * IQR_age
upper_age = Q3_age + 1.5 * IQR_age
print("\nAge Outlier Limits:")
print(lower_age, upper_age)
df = df[(df['Age'] >= lower_age) & (df['Age'] <= upper_age)]
```

```
Age Outlier Limits:
18.0 22.0
```

```
upper_age = Q3_age + 1.5 * IQR_age
print("\nAge Outlier Limits:")
print(lower_age, upper_age)
df = df[(df['Age'] >= lower_age) & (df['Age'] <= upper_age)]
```

```
Age Outlier Limits:
18.0 22.0
```

```
print("\nFinal Cleaned Dataset:")
print(df)
```

```
Final Cleaned Dataset:
    StudentID    Name   Age  Gender      Marks  Attendance            Email
0         101   Aisha  20.0  Female  85.000000        92.0   aisha@gmail.com
1         102     Ali  19.0    Male  78.000000        88.0     ali@gmail.com
2         103    Sara  21.0  Female  95.000000        88.5    sara@gmail.com
5         106   Aisha  20.0  Female  85.000000        92.0   aisha@gmail.com
9         110  Olivia  20.0  Female  72.000000        88.5  olivia@gmail.com
10        111    Liam  19.0    Male  84.363636        85.0    liam@gmail.com
```

Conclusion:

This practical performs:

- Loading and exploring dataset
- Handling missing values logically
- Converting invalid numeric data
- Removing duplicate records
- Detecting and removing outliers using IQR
- Cleaning invalid age, marks, and attendance values

These cessing steps improve data quality before applying machine learning algorithms.

Lab-4

Normalization,Standardization,Discretization,Summary Statistics

Normalization: Normalization is a data preprocessing technique used to scale numerical data into a fixed range, usually between 0 and 1.It ensures that all features contribute equally to the model, especially when values have different units or ranges.

Formula:

$X_{norm} = X - X_{min} X_{max} - X_{min}$

Why used?

- To bring all values to same scale
- Improves machine learning performance
- Used in distance-based algorithms (KNN, clustering)

2. Standardization: Standardization is a technique where data is transformed to have:

- Mean = 0
- Standard Deviation = 1

It is also called **Z-score normalization**.

Formula:

$Z = X - \mu\sigma$

Why used?

- Works well when data has outliers
- Used in regression, SVM, PCA
- Makes data normally distributed (approx.)

3. Discretization: Discretization means converting continuous numerical data into categories or bins**.**

Example: Marks → (Fail, Average, Excellent)

Why used?

- Simplifies data
- Useful in decision trees and rule-based models
- Converts numerical data into meaningful groups

4. Summary Statistics

Summary statistics provide a **quick overview of dataset behavior**.

It includes:

- Mean (average)
- Median (middle value)
- Mode (most frequent value)
- Minimum and Maximum values
- Standard deviation (spread of data)

Why used?

- Understand dataset distribution
- Detect anomalies
- Help in decision making before ML models

```
import pandas as pd
data = {
    'Age': [18, 20, 22, 25, 30, 35, 40, 45],
    'Income': [15000, 18000, 22000, 26000, 30000, 40000, 50000, 60000],
    'Marks': [55, 60, 65, 70, 75, 80, 85, 90]
}

df = pd.DataFrame(data)
print(df)
```

```
   Age  Income  Marks
0   18   15000     55
1   20   18000     60
2   22   22000     65
3   25   26000     70
4   30   30000     75
5   35   40000     80
6   40   50000     85
7   45   60000     90
```

```
min_income = df['Income'].min()
max_income = df['Income'].max()
df['Income_Normalized'] = (df['Income'] - min_income) / (max_income - min_income)
print(df)
```

```
   Age  Income  Marks  Income_Normalized
0   18   15000     55           0.000000
1   20   18000     60           0.066667
2   22   22000     65           0.155556
3   25   26000     70           0.244444
4   30   30000     75           0.333333
5   35   40000     80           0.555556
6   40   50000     85           0.777778
7   45   60000     90           1.000000
```

```
print(df[['Income', 'Income_Normalized']])
```

```
   Income  Income_Normalized
0   15000           0.000000
1   18000           0.066667
2   22000           0.155556
3   26000           0.244444
4   30000           0.333333
5   40000           0.555556
6   50000           0.777778
7   60000           1.000000
```

Task:

- Normalize only the **Income** column manually.
- Compare original vs normalized values.
- What is the mean after standardization?
- Why do some values become negative?
- Create 4 bins for Marks.
- Label them as: Poor, Average, Good, Excellent.
- Find the mean of Income.
- Identify the highest Marks.
- Calculate standard deviation of Age.

```
import pandas as pd
import numpy as np

data = {
    'Income': [20000, 35000, 50000, 65000, 80000],
    'Marks': [45, 60, 75, 85, 95],
    'Age': [18, 20, 22, 24, 26]
}
df = pd.DataFrame(data)
print("Original Dataset:\n", df)
# 1. NORMALIZATION (Income)
df['Income_Normalized'] = (df['Income'] - df['Income'].min()) / (df['Income'].max() - df
print("\nNormalized Income:\n", df[['Income', 'Income_Normalized']])
```

```
# 2. STANDARDIZATION (Income)
df['Income_Standardized'] = (df['Income'] - df['Income'].mean()) / df['Income'].std()
print("\nStandardized Income:\n", df[['Income', 'Income_Standardized']])
print("\nMean after Standardization:", df['Income_Standardized'].mean())
# 3. WHY NEGATIVE VALUES?
print("\nNegative values occur when data is below mean value.")
# 4. DISCRETIZATION (Marks)
bins = [0, 50, 65, 80, 100]
labels = ['Poor', 'Average', 'Good', 'Excellent']
df['Marks_Category'] = pd.cut(df['Marks'], bins=bins, labels=labels)
print("\nMarks after Discretization:\n", df[['Marks', 'Marks_Category']])
```

```
# 5. SUMMARY STATISTICS
# -------------------------
print("\nSummary Statistics:\n")
print(df.describe())
# EXTRA REQUIRED VALUES
# -------------------------
print("\nMean Income:", df['Income'].mean())
print("Highest Marks:", df['Marks'].max())
print("Standard Deviation of Age:", df['Age'].std())
```

```
... Original Dataset:
```

```
print("Standard Deviation of Age:", df['Age'].std(
```

```
Original Dataset:
    Income  Marks  Age
0   20000     45   18
1   35000     60   20
2   50000     75   22
3   65000     85   24
4   80000     95   26

Normalized Income:
    Income  Income_Normalized
0   20000               0.00
1   35000               0.25
2   50000               0.50
3   65000               0.75
4   80000               1.00

Standardized Income:
    Income  Income_Standardized
0   20000            -1.264911
1   35000            -0.632456
2   50000             0.000000
3   65000             0.632456
4   80000             1.264911

Mean after Standardization: 4.4408920985006264e-17

Negative values occur when data is below mean value.

Marks after Discretization:
    Marks Marks_Category
0     45          Poor
1     60       Average
2     75          Good
3     85     Excellent
4     95     Excellent
```

```
1   35000            -0.632456
2   50000             0.000000
3   65000             0.632456
4   80000             1.264911

Mean after Standardization: 4.4408920985006264e-17

Negative values occur when data is below mean value.

Marks after Discretization:
    Marks Marks_Category
0     45          Poor
1     60       Average
2     75          Good
3     85     Excellent
4     95     Excellent

Summary Statistics:

             Income      Marks        Age  Income_Normalized  \
count      5.000000   5.000000   5.000000           5.000000
mean   50000.000000  72.000000  22.000000           0.500000
std    23717.082451  19.874607   3.162278           0.395285
min    20000.000000  45.000000  18.000000           0.000000
25%    35000.000000  60.000000  20.000000           0.250000
50%    50000.000000  75.000000  22.000000           0.500000
75%    65000.000000  85.000000  24.000000           0.750000
max    80000.000000  95.000000  26.000000           1.000000
```

```
Negative values occur when data is below mean value.

Marks after Discretization:
    Marks Marks_Category
0     45          Poor
1     60       Average
2     75          Good
3     85     Excellent
4     95     Excellent

Summary Statistics:

              Income      Marks        Age  Income_Normalized  \
count      5.000000   5.000000   5.000000           5.000000
mean   50000.000000  72.000000  22.000000           0.500000
std    23717.082451  19.874607   3.162278           0.395285
min    20000.000000  45.000000  18.000000           0.000000
25%    35000.000000  60.000000  20.000000           0.250000
50%    50000.000000  75.000000  22.000000           0.500000
75%    65000.000000  85.000000  24.000000           0.750000
max    80000.000000  95.000000  26.000000           1.000000

       Income_Standardized
count         5.000000e+00
mean          4.440892e-17
std           1.000000e+00
min          -1.264911e+00
25%          -6.324555e-01
50%           0.000000e+00
75%           6.324555e-01
max           1.264911e+00

Mean Income: 50000.0
Highest Marks: 95
Standard Deviation of Age: 3.1622776601683795
```

WHAT THIS CODE DOES

- **Normalization (Income):**
  Income column is scaled between 0 and 1 using min-max method.
- **Standardization (Income):**
  Income is converted into Z-scores using mean and standard deviation.
- **Mean after Standardization:**
  Always becomes **0** after transformation.
- **Negative values in Standardization:**
  Values below mean become negative.
- **Discretization (Marks):**
  Marks are converted into categories (Poor, Average, Good, Excellent) using bins.
- **Mean Income:**
  Average value of all Income records.
- **Highest Marks:**
  Maximum value in Marks column.
- **Standard Deviation of Age:**
  Shows how spread out Age values are from the mean.

Lab-5

Market basket analysis,Support,Confidence,Lift calculation manually.

Market Basket Analysis is a data mining technique used to find relationships between items that are frequently purchased together.

It is mainly used in supermarkets and online shopping systems for recommendation and sales improvement.

Support:

Support shows how frequently an item or itemset appears in all transactions.

It helps to identify popular items.

Confidence:

Confidence shows the probability that item B is purchased when item A is purchased.

It measures the strength of a rule.

Lift:

Lift shows how strongly two items are related.

- Lift > 1 → strong relationship
- Lift = 1 → no relationship
- Lift < 1 → weak relationship

Association Rules

Market Basket Analysis works on **association rules** like:

$A \rightarrow B$
This means: If A is bought, then B is likely to be bought.

Example:
If a customer buys **bread**, they may also buy **butter**.

Importance of Market Basket Analysis

- Helps in product recommendation
- Increases sales in supermarkets
- Used in online shopping suggestions (like Amazon)
- Helps in marketing strategies and discount planning

Home Task

Create your own dataset with 5 transactions and:

- **Calculate Support, Confidence, Lift for at least one rule**

```
[4]
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder

[5]
transactions = [
    ['Milk', 'Bread'],
    ['Milk', 'Butter'],
    ['Milk', 'Bread'],
    ['Bread', 'Butter'],
    ['Milk', 'Bread', 'Butter']
]
```

```
[6]
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)
print("One Hot Encoded Dataset:\n")
print(df)

One Hot Encoded Dataset:

   Bread  Butter   Milk
0   True   False   True
1  False    True   True
2   True   False   True
3   True    True  False
4   True    True   True
```

```
[7]
support_milk_bread = df[(df['Milk'] == True) & (df['Bread'] == True)].shape[0] / len(df)
print("Support (Milk & Bread):", support_milk_bread)

Support (Milk & Bread): 0.6

[8]
support_milk = df[df['Milk'] == True].shape[0] / len(df)
confidence = support_milk_bread / support_milk
print("Confidence (Milk → Bread):", confidence)

Confidence (Milk → Bread): 0.7499999999999999

[9]
support_bread = df[df['Bread'] == True].shape[0] / len(df)
lift = confidence / support_bread
print("Lift (Milk → Bread):", lift)

Lift (Milk → Bread): 0.9374999999999998
```

1. What is normalization and why is it important in data preprocessing?

Normalization is a technique that scales data into a fixed range (usually 0 to 1).
It is important because it makes all features equal in scale and improves model performance.

2. What is standardization (Z-score scaling), and how does it differ from normalization?

Standardization transforms data so that:

- Mean = 0
- Standard deviation = 1

Difference:

- Normalization → fixed range (0–1)
- Standardization → no fixed range, uses mean & SD

3. Why is standardization often used before applying machine learning algorithms like k-NN or SVM?

Because these algorithms depend on distance calculations.
Standardization ensures no feature dominates due to large values.

4. What is discretization and when would you use it?

Discretization converts continuous data into categories (bins).
It is used when we want to simplify data or convert it into meaningful groups.

5. Define mean, median, mode, variance, and standard deviation.

- Mean: Average of all values
- Median: Middle value
- Mode: Most frequent value
- Variance: Spread of data
- Standard deviation: Average distance from mean

6. Why are summary statistics important in understanding data?

They help to:

- Understand data quickly
- Detect patterns and outliers
- Summarize large datasets

7. What is market basket analysis? Give a real-world example.

It is a technique to find items that are bought together.

Example:
If a customer buys bread, they may also buy butter in a supermarket.

Lab-6

Apriori algorithm using Python,Frequent itemset generation

Apriori Algorithm

Apriori is a data mining algorithm used to find **frequent itemsets** and generate **association rules** from transaction data.

It works on the principle that:
If an itemset is frequent, then all its subsets must also be frequent.

Frequent Itemset Generation

Frequent itemsets are groups of items that appear together in transactions **above a minimum support threshold**.

Example:
Milk + Bread frequently appear together in shopping baskets.

Steps of Apriori Algorithm

1. Set minimum support value
2. Find frequent individual items
3. Combine items to form itemsets
4. Remove itemsets that do not meet support
5. Repeat until no more frequent itemsets

```
[10]
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
```

```
[11]
transactions = [
    ['Milk', 'Bread'],
    ['Milk', 'Butter'],
    ['Milk', 'Bread'],
    ['Bread', 'Butter'],
    ['Milk', 'Bread', 'Butter']
]
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
```

```
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)
print("One Hot Encoded Data:\n")
print(df)
```

```
One Hot Encoded Data:

   Bread  Butter   Milk
0   True   False   True
1  False    True   True
2   True   False   True
3   True    True  False
4   True    True   True
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarn
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarn
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarn
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarn
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarn
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarn
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarn
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarn
  return datetime.utcnow().replace(tzinfo=utc)
```

```
print("\nAssociation Rules:\n")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift'
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)

Association Rules:

  antecedents consequents  support  confidence    lift
0     (Bread)      (Milk)      0.6        0.75  0.9375
1      (Milk)     (Bread)      0.6        0.75  0.9375
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datet
  return datetime.utcnow().replace(tzinfo=utc)
```

```
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=

print("\nFrequent Itemsets:\n")
print(frequent_itemsets)
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)

Frequent Itemsets:

   support       itemsets
0      0.8        (Bread)
1      0.6       (Butter)
2      0.8         (Milk)
3      0.6  (Bread, Milk)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarnin
  return datetime.utcnow().replace(tzinfo=utc)
```

Lab Tasks

ange min_support to 0.2 and observe results

1. ıerate rules using lift instead of confidence
2. ᵥ transactions and analyze changes Identify:
3. Ad(1. Strongest rule
4. 2. Weakest rule

```
requent_itemsets = apriori(df, min_support=0.2, use_colnames=Tr
rint("Frequent Itemsets (min_support=0.2):")
rint(frequent_itemsets)

equent Itemsets (min_support=0.2):
 support              itemsets
     0.8               (Bread)
     0.6              (Butter)
     0.8                (Milk)
     0.4       (Bread, Butter)
     0.6         (Bread, Milk)
     0.4        (Milk, Butter)
     0.2  (Bread, Milk, Butter)
sr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: 
return datetime.utcnow().replace(tzinfo=utc)
```

```
es_lift = association_rules(frequent_itemsets, metric="lift", min_threshold=1)

nt("Rules using Lift:")
nt(rules_lift[['antecedents', 'consequents', 'support', 'confidence', 'lift']]

/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
urn datetime.utcnow().replace(tzinfo=utc)
/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
urn datetime.utcnow().replace(tzinfo=utc)
/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
urn datetime.utcnow().replace(tzinfo=utc)
/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
urn datetime.utcnow().replace(tzinfo=utc)
/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
urn datetime.utcnow().replace(tzinfo=utc)
/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
urn datetime.utcnow().replace(tzinfo=utc)
/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
urn datetime.utcnow().replace(tzinfo=utc)
s using Lift:
/ DataFrame
ns: [antecedents, consequents, support, confidence, lift]
:: []
/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
urn datetime.utcnow().replace(tzinfo=utc)
```

```
ansactions = [
   ['Milk', 'Bread'],
   ['Milk', 'Butter'],
   ['Milk', 'Bread'],
   ['Bread', 'Butter'],
   ['Milk', 'Bread', 'Butter'],
   ['Milk', 'Bread', 'Jam'],    # NEW
   ['Bread', 'Jam']             # NEW

r/local/lib/python3.12/dist-packages/jupyter_client/session.
eturn datetime.utcnow().replace(tzinfo=utc)
r/local/lib/python3.12/dist-packages/jupyter_client/session.
eturn datetime.utcnow().replace(tzinfo=utc)
r/local/lib/python3.12/dist-packages/jupyter_client/session.
eturn datetime.utcnow().replace(tzinfo=utc)
r/local/lib/python3.12/dist-packages/jupyter_client/session.
```

[18]

```python
strongest_rule = rules_lift.sort_values(by='lift', ascending=False).head(1)

print("Strongest Rule:")
print(strongest_rule[['antecedents','consequents','lift']])
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
Strongest Rule:
Empty DataFrame
Columns: [antecedents, consequents, lift]
Index: []
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
```

[19]

```python
weakest_rule = rules_lift.sort_values(by='lift', ascending=True).head(1)

print("Weakest Rule:")
print(weakest_rule[['antecedents','consequents','lift']])
```

```
Weakest Rule:
Empty DataFrame
Columns: [antecedents, consequents, lift]
Index: []
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is
  return datetime.utcnow().replace(tzinfo=utc)
```

Lab-7

Implement Naïve Bayes and KNN

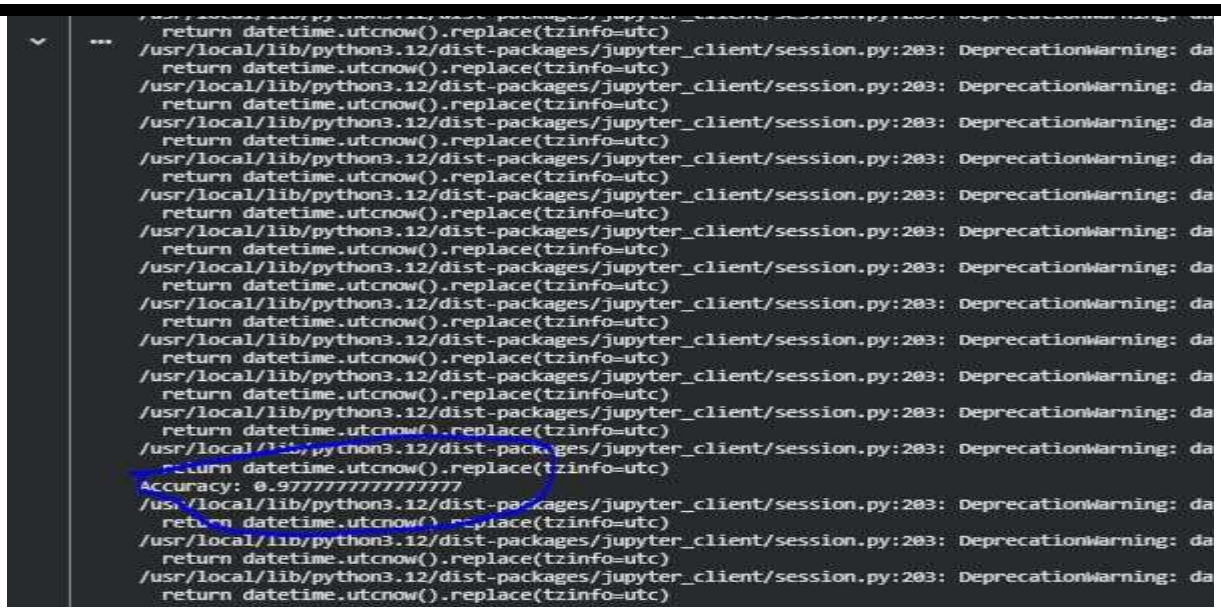
Naïve Bayes

Naïve Baye is a probabilistic algorithm based on Bayes theorem.
It assume features are independent.
all

Used for:

- tection
- Spam ssification
- Text

```
m sklearn.datasets import load_iris
m sklearn.model_selection import train_test_split
m sklearn.naive_bayes import GaussianNB
m sklearn.metrics import accuracy_score

oad dataset
a = load_iris()
 data.data
 data.target

plit dataset
rain, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

odel
el = GaussianNB()
el.fit(X_train, y_train)

rediction
red = model.predict(X_test)

ccuracy
nt("Accuracy:", accuracy_score(y_test, y_pred))
```



KNN (K-Nearest Neighbors)

KNN is a distance-based ithm.
It classifies data based on neighbors.
nearest

Used for:

- Classification
- Recommendation systems

```
rnings('ignore')
sets import load_iris
l_selection import train_test_split
hbors import KNeighborsClassifier
ics import accuracy_score

)




y_train, y_test = train_test_split(
ze=0.3, random_state=42


sClassifier(n_neighbors=5)
, y_train)

edict(X_test)
UTPUT
, round(accuracy_score(y_test, y_pred), 4))
```

```
print('Accuracy:', round(accuracy_score(y_test, y_pred

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: Depreca
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: Depreca
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: Depreca
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: Depreca
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: Depreca
  return datetime.utcnow().replace(tzinfo=utc)
Accuracy: 1.0
```

KNN vs Naïve Bayes Comparison Table

Feature	K-Nearest Neighbors (KNN)	Naïve Bayes
Type	Instance-based (Lazy learning)	Probabilistic model
Working Principle	Based on distance (nearest neighbors)	Based on probability (Bayes theorem)
Training Time	No training (fast)	Fast training
Prediction Time	Slow (checks all data)	Fast
Accuracy	Depends on data & K value	Good for small datasets
Feature Assumption	No assumption	Assumes features are independent
Best for	Small datasets, simple classification	Text classification, spam detection
Sensitivity	Sensitive to scaling & noise	Less sensitive to noise
Memory Usage	High (stores full dataset)	Low
Formula	Distance (Euclidean, etc.)	P(A

Lab-8

K-Means clustering

K-Means clustering is a popular **unsupervised machine learning algorithm** used to group similar data points into clusters. It is widely used in data mining, image segmentation, customer segmentation, and pattern recognition.

What it does?

K-Means divides a dataset into **K number of clusters**, where each data point belongs to the cluster with the nearest mean (called the *centroid*).

Basic Idea

The algorithm tries to:

- Keep points in the same cluster **similar to each other**
- Keep different clusters **as distinct as possible**

Steps of K-Means Algorithm

1. **Choose K**

   Decide how many clusters you want (e.g., K = 3)

2. **Initialize Centroids**

   Randomly select K points as initial centroids

3. **Assign Points to Clusters**

   Each data point is assigned to the nearest centroid (using distance like Euclidean distance)

4. **Update Centroids**

   Calculate new centroid by taking the mean of all points in each cluster

5. **Repeat**

   Repeat steps 3 and 4 until centroids stop changing (convergence)

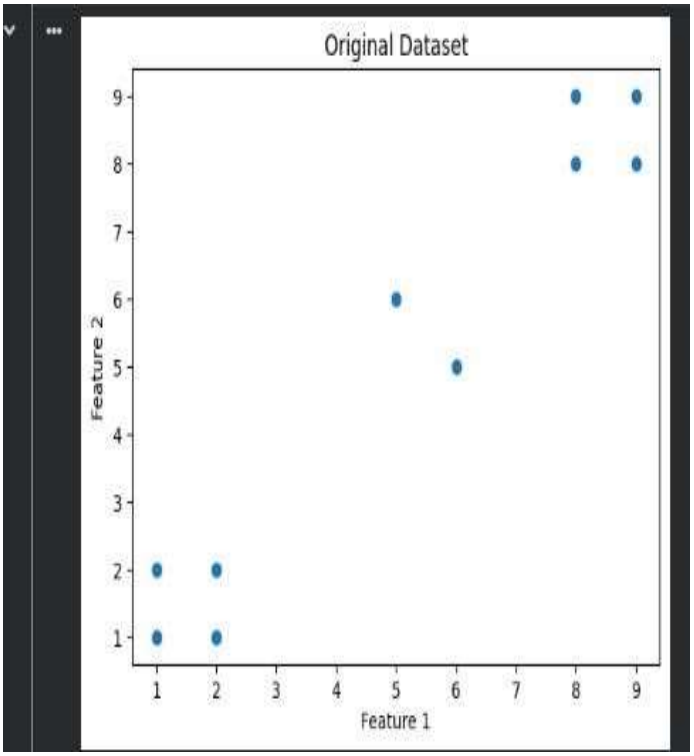
TASK:1

Part 1: K-Means Clustering (Unsupervised Learning)

Task 1.1:

- Create a dataset (at least 8–10 data points with 2 features)
- Plot the data using matplotlib

```
[29]
✓ 0s
import matplotlib.pyplot as plt
import numpy as np
# 10 data points (2 features)
X = np.array([
    [1, 2],
    [2, 1],
    [1, 1],
    [2, 2],
    [8, 8],
    [9, 8],
    [8, 9],
    [9, 9],
    [5, 6],
    [6, 5]
])
# Plot data
plt.scatter(X[:, 0], X[:, 1])
plt.title("Original Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Task 1.2:

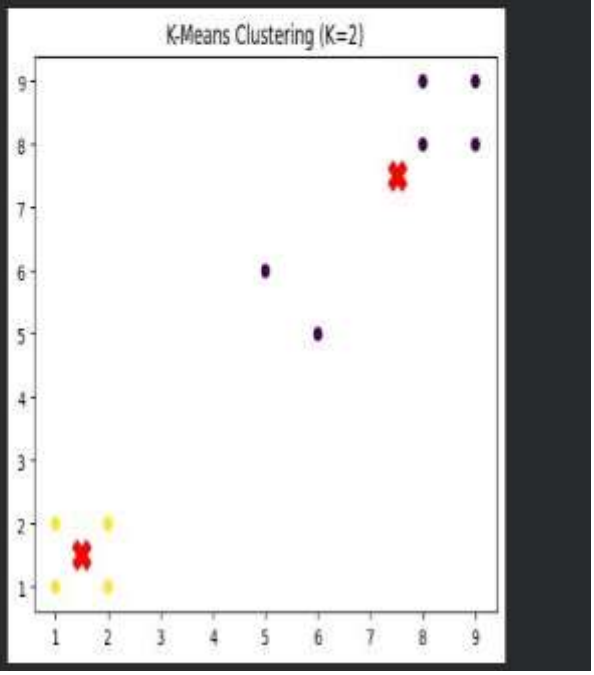
- Apply K-Means algorithm with:
  - K = 2
  - K = 3

```
[30]
✓ 0s
from sklearn.cluster import KMeans
# K = 2
kmeans2 = KMeans(n_clusters=2, random_state=42, n_init=10)
labels2 = kmeans2.fit_predict(X)
# K = 3
kmeans3 = KMeans(n_clusters=3, random_state=42, n_init=10)
labels3 = kmeans3.fit_predict(X)
```

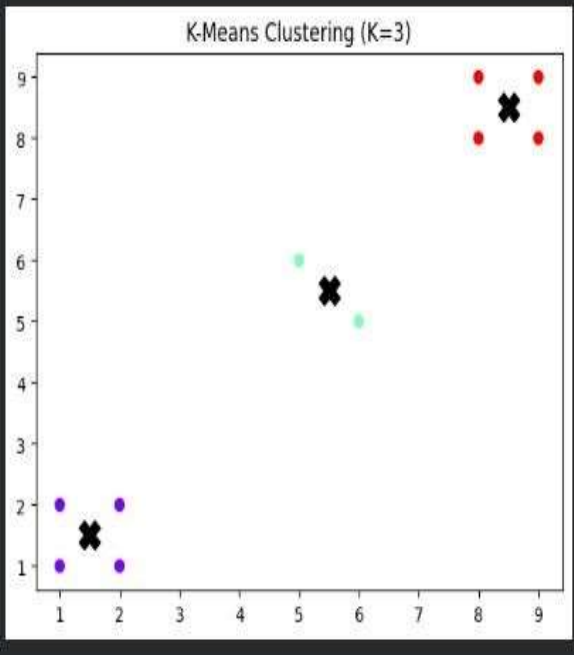
Task 1.3:

- Plot clusters with different colors
- Mark centroids clearly

```
plt.scatter(X[:, 0], X[:, 1], c=labels2, cmap='viridis')
plt.scatter(kmeans2.cluster_centers_[:, 0],
            kmeans2.cluster_centers_[:, 1],
            color='red', marker='X', s=200)
plt.title("K-Means Clustering (K=2)")
plt.show()
```




```
plt.scatter(X[:, 0], X[:, 1], c=labels3, cmap='rainbow'
plt.scatter(kmeans3.cluster_centers_[:, 0],
            kmeans3.cluster_centers_[:, 1],
            color='black', marker='X', s=200)

plt.title("K-Means Clustering (K=3)")
plt.show()
```



Task 1.4:

- **Answer:**

How does changing K affect clustering?

When we change K:

- The number of clusters increases or decreases
- Data grouping changes
- Small K → large general groups
- Large K → more detailed but sometimes unnecessary clusters

Which K is better and why?

- If data has **2 natural groups → K=2 is better**
- If data has **3 natural patterns → K=3 is better**

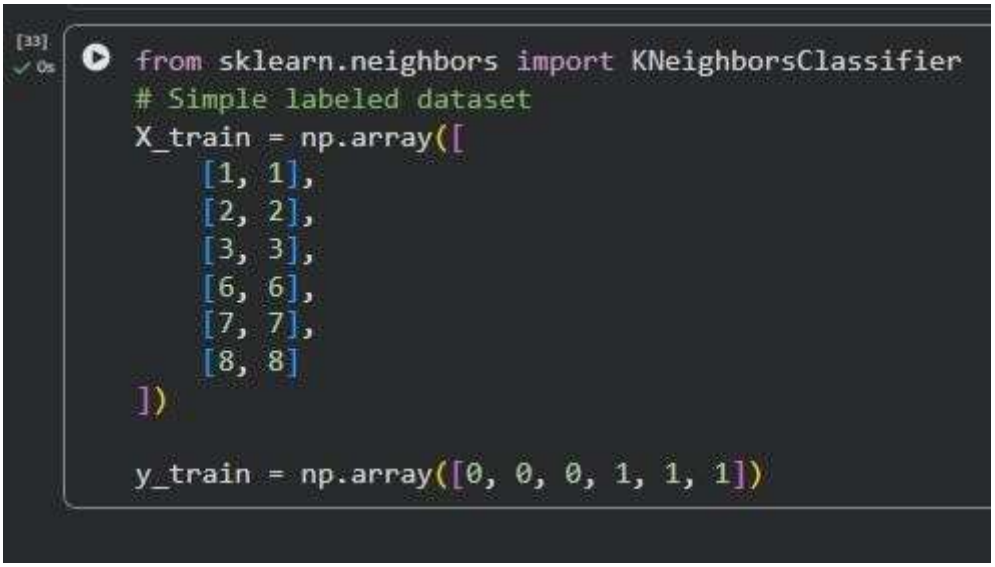
Best K is the one that:

- Matches real structure of data
- Gives clear separation
- Has low within-cluster distance (WCSS)

Part 2: K-Nearest Neighbors (KNN)

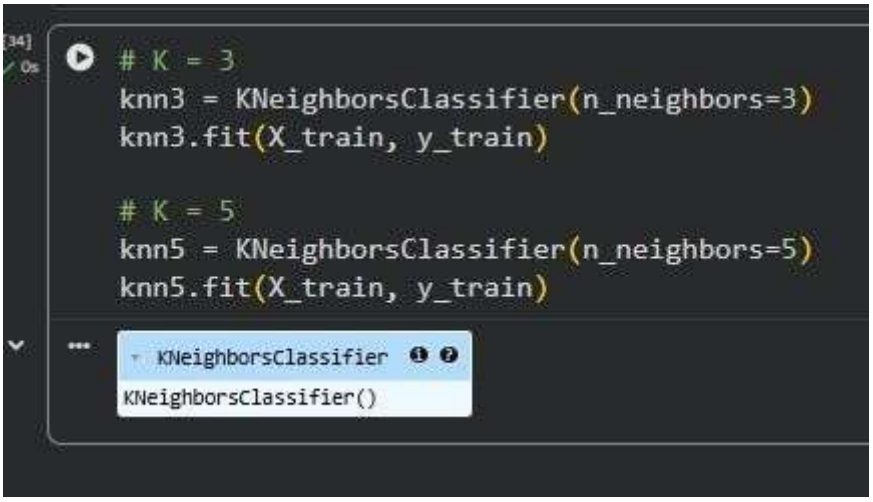
Task 2.1:

- Create a simple labeled dataset (at least 6–10 samples)



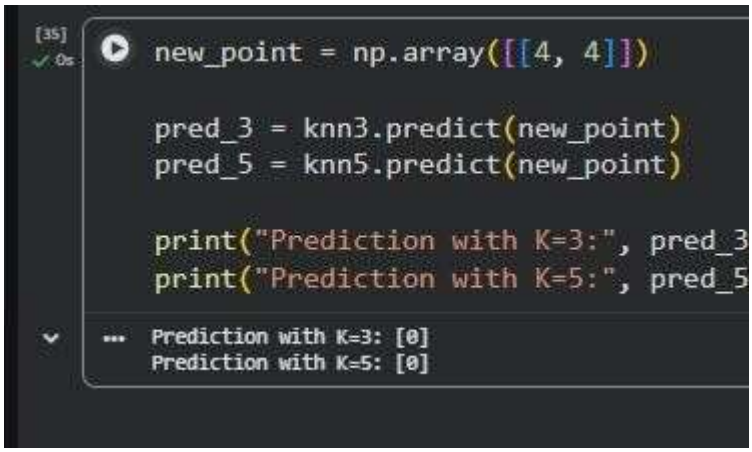
Task 2.2:

- Train a KNN model with:
  - K = 3
  - K = 5



Task 2.3:

- Predict the class of a new data point



Task 2.4:

- Answer:

What happens when K is too small or too large?

- **K too small (e.g., K=1):**
  - Model becomes noisy
  - Sensitive to outliers
  - Overfitting happens
- **K too large:**
  - Model becomes too smooth
  - Different classes may mix
  - Underfitting happens

Why is KNN called a lazy learner?

KNN is called a **lazy learner** because:

- It does NOT learn a model during training
- It simply stores the dataset
- It performs calculations only during prediction time

Comparison & Analysis

- Difference between K-Means and KNN
- Real-life applications of each
- Advantages and limitations

Feature	K-Means Clustering	K-Nearest Neighbors (KNN)
Type	Unsupervised Learning	Supervised Learning
Data	Unlabeled data	Labeled data
Purpose	Grouping/Clustering	Classification / Prediction
Idea	No training labels required	Finds nearest neighbors
Training	Cluster ID	Stores training data
Output	Lazy grouping but iterative	Class label
Learning Type	Finds cluster centers (centroids)	Fully lazy learner

Real-Life Applications

K-Means Applications

- Customer segmentation (marketing)
- Image compression
- Grouping similar documents/articles
- Organizing social media users ● Market basket analysis

Example:
Online shopping websites group customers based on buying behavio

KNN Applications

- Spam email detection
- Medical diagnosis (disease prediction)
- Face recognition
- Handwriting recognition
- Recommendation systems

Advantages

- Simple and fast algorithm
- Works well for large datasets
- Easy to implement
- Efficient for clear cluster shapes

Limitations:

- Must choose K manually
- Sensitive to initial centroids

- Poor performance on non-spherical clusters

- Affected by outliers

Lab-9

Hierarchical clustering

Hierarchical Clustering is an **unsupervised learning algorithm** used to group similar data points into a hierarchy of clusters. Unlike K-Means, it does **not require the value of K in advance**.

What it does

It builds a **tree-like structure of clusters** called a **dendrogram**, where:

- Each data point starts as its own cluster
- Clusters are gradually merged (or split)

Types of Hierarchical Clustering

Agglomerative (Bottom-Up oach)

Most commonly used.

Steps:

- Start with each data point as a single cluster
- Find closest clusters
- Merge them step by step
- Continue until all points one cluster become

This is the **most used method in ractice** data)

Divisive (Top-Down Approach)

Opposite of agglomerative. oint is

Steps:

- Start with one big cluster (all
- Split into smaller clusters eparate
- Continue splitting until each

Dendrogram (Important Concept) at shows:

A **dendrogram** is a tree diagram

- How clusters are merged by cutting the dendrogram at a specific level.
- Distance between clusters

You decide the number of clusters **is measured)**

Distance Methods (How similarity

Common distance metrics:

- Euclidean distance (most common)
- Manhattan distance
- Cosine similarity

Linkage Criteria (How clusters are merged)

- **Single Linkage** → minimum distance between clusters
- **Complete Linkage** → maximum distance between clusters
- **Average Linkage** → average distance between clusters

Advantages

- No need to predefine K
- Easy to understand (tree structure)
- Good for small datasets
- Works well for hierarchical data

Limitations

- Very slow for large datasets
- Cannot undo merges (greedy approach)
- Sensitive to noise and outliers ● High memory usage

Real-Life Applications

- Gene sequence analysis (bioinformatics)
- Document clustering (news articles)
- Customer grouping
- Social network analysis ● Image segmentation

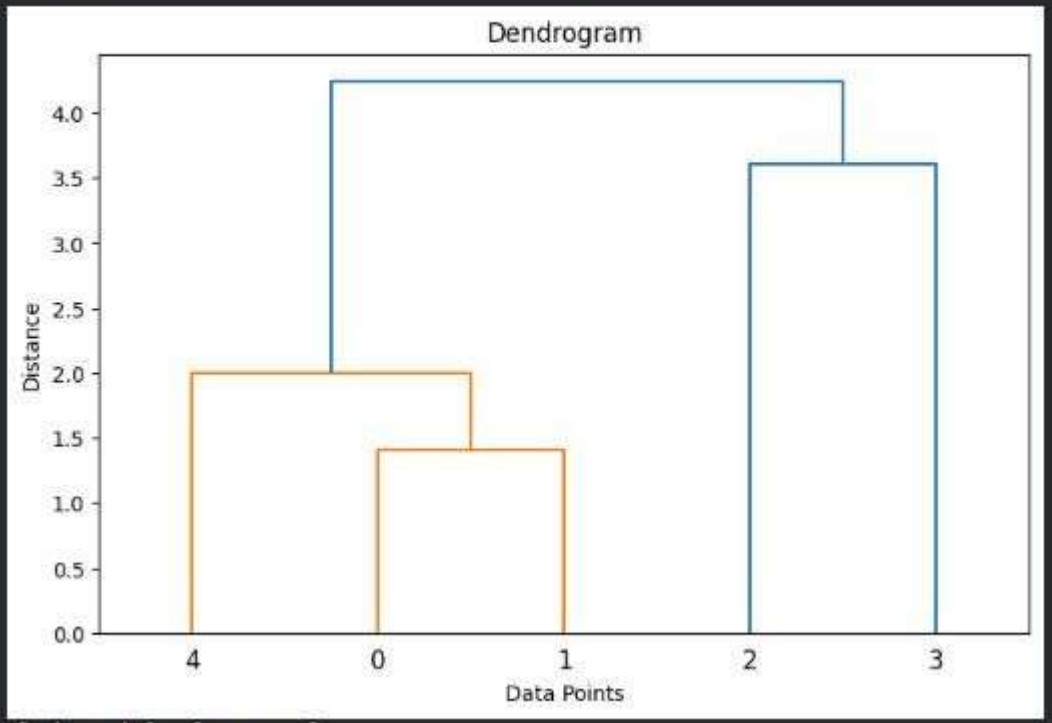
Simple Example

Imagine students grouped based on marks:

- First, each student is separate
- Then similar students are grouped
- Finally, all groups merge into a hierarchy

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

data = np.array([[1,2], [2,3], [5,6], [8,8], [1,0]])
linkage_matrix = linkage(data, method='single') # methods: single, complete, average
plt.figure(figsize=(8,5))
dendrogram(linkage_matrix)
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
from scipy.cluster.hierarchy import fcluster

clusters = fcluster(linkage_matrix, t=3, criterion='maxclust')
print("Cluster Labels:", clusters)
```

```python
clusters = fcluster(linkage_matrix, t=3, criterion='maxclust')
print("Cluster Labels:", clusters)
```




```
Cluster Labels: [1 1 2 3 1]
```

Lab-10

Distance-based outliers,Isolation Forest,Practical datasets

1. Distance-Based Outliers

Outliers are data points that are **far away from most other points**.
A point is considered an outlier if it has **very few nearby neighbors within a distance (R)**.

Idea: "Far from others = Outlier"

2. Isolation Forest

Isolation Forest is an **anomaly detection algorithm**.
It works by **randomly splitting data**. Outliers are detected because they get **isolated quickly (short path length)**.

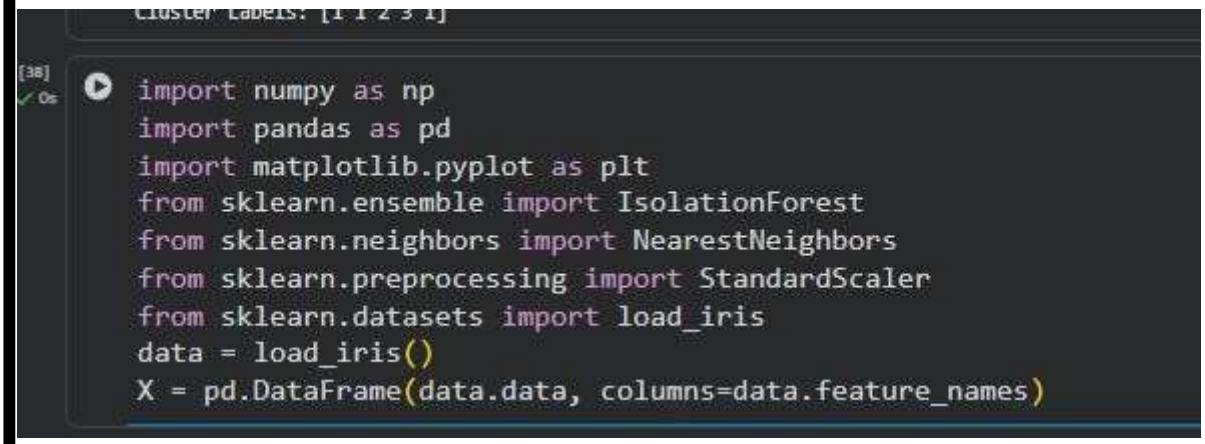
Idea: "Easy to isolate = Outlier"

3. Practical Datasets

Common datasets used for outlier detection:

- Iris dataset
- Credit Card Fraud dataset
- Wine Quality dataset
- Breast Cancer dataset

Used to test anomaly detection algorithms in real life.



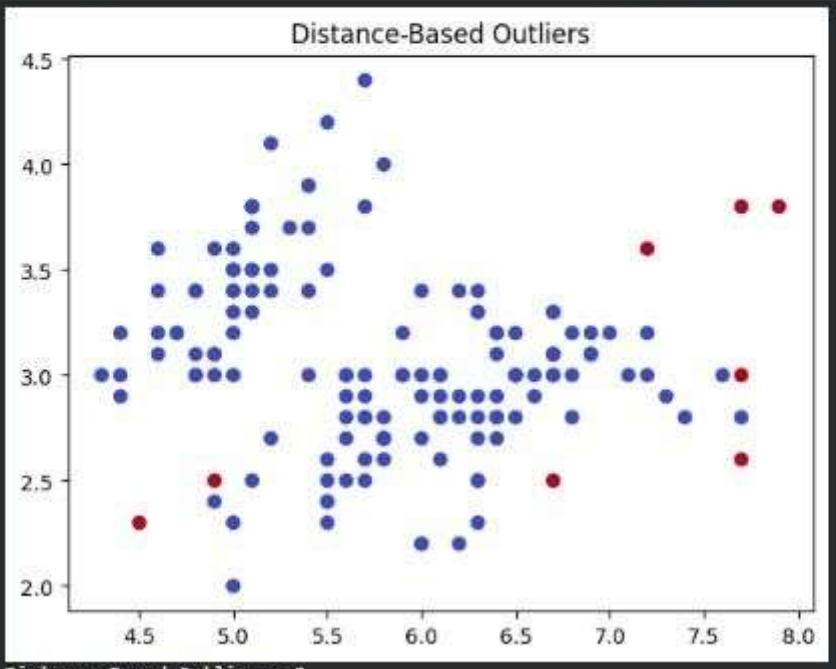
```
[39]
✓ 0s
```

```
k = 5
nbrs = NearestNeighbors(n_neighbors=k)
nbrs.fit(X)
distances, indices = nbrs.kneighbors(X)
outlier_scores = distances.mean(axis=1)
X['Outlier_Score'] = outlier_scores
threshold = np.percentile(outlier_scores, 95)
X['Outlier'] = outlier_scores > threshold
print(X.head())
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Outlier_Score  Outlier
0       0.104853    False
1       0.119494    False
2       0.183104    False
3       0.156636    False
4       0.125851    False
```

```
[41]
✓ 0s
```

```
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=X['Outlier'], cmap='coolwar
plt.title("Distance-Based Outliers")
plt.show()
print("Distance-Based Outliers:", X['Outlier'].sum())
print("Isolation Forest Outliers:", X['Iso_Outlier'].sum())
```




```
Distance-Based Outliers: 8
Isolation Forest Outliers: 8
```

Lab-11

FP-Tree construction, FP-Growth implementation using library

FP-Tree Construction:

FP-Tree (Frequent Pattern Tree) is a **compressed tree structure** used to store frequent items from a dataset.

It is used in **frequent itemset mining** without generating candidate sets.

Working Idea:

- Scan dataset and find frequent items
- Remove infrequent items
- Sort items by frequency
- Build a tree where **common prefixes are shared**

Key Points:

- Stores data in compact form
- Each node shows item and count
- Same items share paths in tree

Advantage:

- Reduces database scans
- Saves memory compared to Apriori

Limitation:

- Complex to build
- Can become large for dense data

FP-Growth Algorithm (Theory)

FP-Growth is an algorithm that finds **frequent itemsets using FP-Tree**

Steps:

1. Build FP-Tree from dataset
2. Extract conditional pattern base
3. Build conditional FP-Trees
4. Generate frequent itemsets

Key Idea:

"No candidate generation needed"

Advantages:

- Faster than Apriori
- Requires only 2 database scans
- Efficient for large datasets

Limitations:

- Tree construction is complex
- Memory usage can increase

ASSIGNMENT terms:

Question 1: Define the following

1. Frequent itemset:

A **frequent itemset** is a group of items that appear together many times in a transaction database. of items is greater than or equal to the minimum support value, then that set of items is called a frequent itemset. are used in data mining to discover customer buying patterns and relationships between products.

If the occurrence of Bread and Milk together is frequent, it means many customers buy Bread and Milk together, then {Bread, Milk} becomes a frequent itemset.

Example:If an item exists in the dataset.

2. Support Count:

Support count is the number of transactions in which a particular item or itemset appears. frequently

Count(X)=Number of transactions containing X

Support count

It measures how

Formula::
Support

Example:

Suppose the transactions are:

- T1 = Bread Butter
- T2 = Bread Milk,
- T3 = Bread of **Bread** is **3** because it appears in all three transactions. of **Milk** is **2** because it appears in T1 and T3.

3. FP-Tree:

An **FP-Tree (Frequent Pattern Tree)** is a tree-like data structure used to store transactions in a compact form.It is used in the FP-Growth algorithm to find frequent itemsets efficiently.

The support count

The support count

transaction data
in frequent
itemsets

The FP-Tree stores common items together in shared branches, which reduces memory usage and processing time.

Features of FP-Tree:

- Compact representation of data
- Stores only frequent items
- Reduces repeated database scanning
- Faster than traditional methods

Question 2:

Given Transaction Database

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Diaper, Beer, Eggs
T3	Milk, Bread, Diaper, Beer
T4	Bread, Milk, Diaper, Beer
T5	Milk, Bread, Diaper, Cola

Task 1: Calculate Support Count of Each Item:

Support count means the number of transactions in which an item appears.

Item	Transactions	Support Count
Milk	T1, T3, T4, T5	4
Bread	T1, T2, T3, T4, T5	5
Butter	T1	1
Diaper	T2, T3, T4, T5	4
Beer	T2, T3, T4	3
Eggs	T2	1
Cola	T5	1

Task 2: Minimum Support = 3

Minimum support means an item must appear at least 3 times to be considered frequent.

Frequent Items

Item	Support Count
Bread	5
Milk	4
Diaper	4
Beer	3

Infrequent Items Removed

- Butter
- Eggs
- Cola

These items are removed because their support count is less than 3

Task 3: Transactions After Removing Infrequent Items

Transaction ID	Remaining Items
T1	Milk, Bread
T2	Bread, Diaper, Beer
T3	Milk, Bread, Diaper, Beer
T4	Bread, Milk, Diaper, Beer
T5	Milk, Bread, Diaper

Task 4: Arrange Items According to Frequency

Items are arranged in descending order of support count.

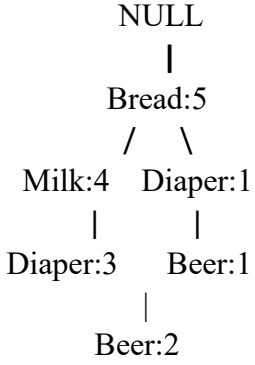
Item	Support
Bread	5
Milk	4
Diaper	4
Beer	3

Task 4: Arrange Items According to Frequency

Items are arranged in descending order of support count.

Item	Support
Bread	5
Milk	4
Diaper	4
Beer	3

Manual FP-Tree Construction



Explanation of FP-Tree

- Bread appears in all transactions, so it becomes the main node.
- Milk appears four times after Bread.
- Diaper appears after Milk in many transactions.
- Beer appears with Diaper in some transactions.

The FP-Tree stores common paths together, which saves memory and makes mining faster.

Question 3:

FP-Growth Implementation Using Python
:

```
[1]
!pip install mlxtend
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth
dataset = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Bread', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Milk', 'Bread', 'Diaper', 'Cola']
]
te = TransactionEncoder()
te_array = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_array, columns=te.columns_)
frequent_itemsets = fpgrowth(df, min_support=0.6, use_colnames=True)
print(frequent_itemsets)
```

```
Requirement already satisfied: scikit-learn>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from mlx
Requirement already satisfied: matplotlib>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from mlxte
Requirement already satisfied: joblib>=0.13.2 in /usr/local/lib/python3.12/dist-packages (from mlxtend)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplo
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matpl
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matpl
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplot
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplo
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from ma
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from sc
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateuti
    support               itemsets
0       1.0                (Bread)
1       0.8                 (Milk)
2       0.8               (Diaper)
3       0.6                 (Beer)
4       0.8          (Bread, Milk)
5       0.8        (Bread, Diaper)
6       0.6         (Milk, Diaper)
7       0.6  (Bread, Diaper, Milk)
8       0.6         (Beer, Diaper)
9       0.6          (Beer, Bread)
10      0.6  (Beer, Bread, Diaper)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.date
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.date
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.date
  return datetime.utcnow().replace(tzinfo=utc)
```

- **What is the difference between Apriori and FP-Growth?**

Apriori and FP-Growth are both used to find frequent itemsets, but they work in different ways. Apriori generates many possible combinations of items (called candidate itemsets) and then checks which ones are frequent. Because of this, it needs to scan the database again and again, which makes it slow for large datasets. On the other hand, FP-Growth does not generate candidate itemsets. Instead, it builds a compact structure called an FP-Tree and extracts frequent patterns directly from it. This makes FP-Growth more efficient and faster compared to Apriori.

- **Why is FP-Growth considered faster than Apriori?**

FP-Growth is faster because it avoids generating unnecessary candidate itemsets. It scans the database only a few times and stores the data in the form of an FP-Tree. This tree structure keeps common data together, which reduces repetition and saves time. In contrast, Apriori repeatedly scans the database and checks many combinations, which increases processing time. Therefore, FP-Growth performs better, especially when the dataset is large.

- **What is the purpose of the FP-Tree?**

The main purpose of the FP-Tree is to store transaction data in a compressed and organized way. Instead of storing each transaction separately, it combines similar transactions into shared paths. This reduces memory usage and avoids repeated calculations. The FP-Tree helps in quickly finding frequent itemsets by making the data mining process more efficient and structured.

- **What does minimum support mean?**

Minimum support is a threshold value that determines whether an item or itemset is important or not. It defines the minimum number of times an item must appear in the dataset to be considered frequent. If an item appears less than this value, it is ignored. Minimum support helps remove less useful or rare items and focuses only on important patterns in the data

Lab-12

Introduction to Sentiment Analysis

What is Sentiment Analysis?

Sentiment Analysis is a technique used in Natural Language Processing (NLP) to identify and classify emotions or opinions expressed in a piece of text. It helps determine whether the sentiment of the text is **Positive**, **Negative**, or **Neutral**.

It is widely used in:

- Product review analysis
- Social media monitoring
- Customer feedback systems
- Movie and hotel review analysis
- Business decision making

For example:

- "This product is amazing" → Positive
- "I hate this service" → Negative
- "The product is okay" → Neutral

Code Implementation of Sentiment Analysis Using Python and VADER

What is VADER?

VADER (Valence Aware Dictionary and Sentiment Reasoner) is a lexicon and rule-based sentiment analysis tool specially designed for social media texts and short reviews.It is simple, fast, and gives accurate sentiment scores.

```
[46]
!pip install vaderSentiment
import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
reviews = [
    "This mobile phone is excellent",
    "The service was very bad",
    "The product is average",
    "I am happy with the purchase",
    "The delivery was delayed and disappointing"]
results = []
for review in reviews:
    score = analyzer.polarity_scores(review)
    compound = score['compound']
    if compound >= 0.05:
        sentiment = "Positive"
    elif compound <= -0.05:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"
    results.append([review, sentiment])
df = pd.DataFrame(results, columns=['Review', 'Sentiment'])
print(df)
```

```
[46]
    results.append([review, sentiment])
df = pd.DataFrame(results, columns=['Review', 'Sentiment'])
print(df)

Collecting vaderSentiment
  Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl.metadata (572 bytes)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from vaderSentiment) (2.32
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from reques
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSe
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->va
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->va
Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl (125 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 126.0/126.0 kB 3.6 MB/s eta 0:00:00
Installing collected packages: vaderSentiment
Successfully installed vaderSentiment-3.3.2
                                     Review Sentiment
0            This mobile phone is excellent  Positive
1                 The service was very bad  Negative
2                   The product is average   Neutral
3              I am happy with the purchase  Positive
4  The delivery was delayed and disappointing  Negative
```